

Logic Synthesis & Verification, Fall 2026

National Taiwan University

Programming Assignment 1

Exercises 1-3 are due on 9/25 23:59; Exercise 4 is due on 10/9 23:59.
Please submit Exercises 1 and 4 on GitHub, and Exercises 2 and 3 on NTU Cool.

Submission Guidelines. For Exercises 2 and 3, please put the required items under `lsv/pa1/`, i.e., this folder. Compress the `lsv/pa1/` folder as a single `.tgz` file and submit it on NTU cool. For Exercise 4, please develop your code under `src/ext-lsv`. You are asked to submit your assignments by creating pull requests to your own branch. To avoid plagiarism, please push files and create pull requests one hour before the deadline. Please see the GitHub page (<https://github.com/NTU-ALComLab/LSV-PA>) for more details.

1 [Getting Familiar with GitHub] (0%)

- (a) Open the GitHub page and check whether there is your own branch named by your student ID number. If you cannot find your own branch or your branch is inconsistent with the `master` branch, please contact TA.
- (b) Fork the repository to your personal GitHub account. You will need to develop your programs on your forked repository.
- (c) Edit `participants-id.csv` under `lsv/admin/` to register your student ID, name, and GitHub account. Send a pull request to the `master` branch after you finish it. Note that this is the only time you send a pull request to the `master` branch. In the following, you are asked to send pull requests to your own branch.

2 [Using ABC] (10%)

- (a) Create a BLIF file named `mul.blif` to represent a two-bit unsigned multiplier $Y = A * B$, where $A = (a1\ a0)$ and $B = (b1\ b0)$ are two-bit unsigned integers, and $Y = (y3\ y2\ y1\ y0)$ is a four-bit unsigned integer. You can find the BLIF manual in <http://www.eecs.berkeley.edu/~alanmi/publications/other/blif.pdf>.
- (b) Perform the following steps to practice using ABC with the two-bit unsigned multiplier example. Screenshot the results after running the commands and put them in your report.
 - 1. read the BLIF file into ABC (command `"read"`)
 - 2. check statistics (command `"print_stats"`)
 - 3. visualize the network structure (command `"show"`)
 - 4. convert to AIG (command `"strash"`)
 - 5. visualize the AIG (command `"show"`)

6. convert to BDD (command “collapse”)
7. visualize the BDD (command “show_bdd -g”; note that “show_bdd” only shows the first PO; option “-g” can be applied to show all POs)

3 [ABC Boolean Function Representations] (10%)

In ABC, there are different ways to represent Boolean functions.

- (a) Compare the following differences with the two-bit unsigned multiplier example. Screenshot the results and briefly describe your findings in your report.
 1. logic network in AIG (by command “aig”) vs. structurally hashed AIG (by command “strash”)
 2. logic network in BDD (by command “bdd”) vs. collapsed BDD (by command “collapse”)
- (b) Given a structurally hashed AIG, find a sequence of ABC commands to convert it to a logic network with node function expressed in sum-of-products (SOP). Use the two-bit unsigned multiplier example to test your command sequence, screenshot the results, and put them in your report.

4 [Programming ABC] (80%)

In this problem, you are asked to write your own procedures and integrate them into ABC, so that your self-defined commands can be executed within ABC. You may need to trace some source codes in ABC to understand the data structures and function usages.

Hint 1: You may refer to `src/base/abc/abc.h` to find definitions of some important variables, functions, iterators, etc.

Hint 2: You may use the command “`grep -R <keyword>`” to find whether a certain keyword appears in the source codes.

Hint 3: You may refer to the CUDD package website to find some useful BDD operations.

4.1 k-feasible Cut Truth Table Generation

Write a procedure in the ABC environment to enumerate all k-feasible cuts of every node on an AIG, and generate corresponding truth table for each cut. For this assignment, it is sufficient to enumerate and print only cuts rooted at internal AIG nodes. Cuts rooted at primary inputs or primary outputs do not need to be printed. Integrate this procedure into ABC (under `src/ext-lsv/`), so that after reading in a circuit (by command “`read`”) and transforming it into AIG (by command “`strash`”), running the command “`lsv_cut_tt`” would invoke your code.

The command should have the following format.

```
lsv_cut_tt <k>
```

The output should have the following format.

```

<node_1>: <cut_1>: <truth table>
<node_1>: <cut_2>: <truth table>
...
<node_2>: <cut_1>: <truth table>
...

```

where **<node_i>** is a node ID, and each **<cut_j>** after **<node_i>** is a k -feasible cut for **<node_i>**. **<cut_i>** should be a sequence of node IDs, sorted in ascending order, separated by space characters.

<node_1> **<node_2>** **<node_3>** ...

<truth table> should be the hexadecimal representation of the cut's truth table. For a cut with k inputs, the truth table should be constructed by evaluating the cut for all 2^k input assignments in binary counting order, from 00...0 to 11...1. The input variables should follow the same order as the node IDs in **<cut_i>**. The output bit corresponding to input assignment 00...0 should be the least significant bit (LSB), and the output bit corresponding to 11...1 should be the most significant bit (MSB). The resulting 2^k -bit sequence should then be converted to hexadecimal. Note that in our test cases, k ranges from 2 to 6. Therefore, each truth table contains at most 64 bits. Consequently, the entire truth table can be stored in a 64-bit unsigned integer, such as uint64_t.

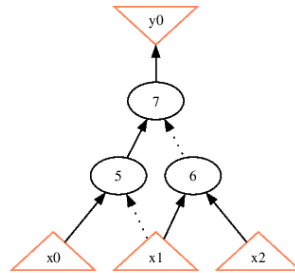


Fig. 1. A simple AIG

For example, given the AIG in Fig. 1, where ID of x_0 , x_1 , x_2 are 1, 2, 3 respectively, a command to compute truth table of 3-feasible cuts and the corresponding output should look like:

```

abc 01> lsv_cut_tt 3
5: 5: 2
5: 1 2: 4
6: 6: 2
6: 2 3: 8

```

```

7: 7: 2
7: 5 6: 4
7: 2 3 5: 2A
7: 1 2 6: 10
7: 1 2 3: 30

```

4.2 k-feasible Cut BDD Generation

Write a procedure in the ABC environment to enumerate all k-feasible cuts of every node on an AIG, and generate corresponding ROBDD for each cut, and print the BDD size. Note that the BDD decision order must be consistent with AIG node IDs. Variables with smaller node IDs must be tested earlier, i.e., closer to the BDD root, than variables with larger node IDs. For this assignment, it is sufficient to enumerate and print only cuts rooted at internal AIG nodes. Cuts rooted at primary inputs or primary outputs do not need to be printed. Integrate this procedure into ABC (under `src/ext-lsv/`), so that after reading in a circuit (by command “`read`”) and transforming it into AIG (by command “`strash`”), running the command “`lsv_cut_bddsize`” would invoke your code.

The command should have the following format.

```
lsv_cut_bddsize <k>
```

The output should have the following format.

```

<node_1>: <cut_1>: <BDD size>
<node_1>: <cut_2>: <BDD size>
...
<node_2>: <cut_1>: <BDD size>
...

```

where `<node_i>` is a node ID, and each `<cut_j>` after `<node_i>` is a k-feasible cut for `<node_i>`. `<cut_i>` should be a sequence of node IDs, sorted in ascending order, separated by space characters.

```
<node_1> <node_2> <node_3> ...
```

For example, given the AIG in Fig. 1, a command to compute ROBDD size of 3-feasible cuts and the corresponding output should look like:

```

abc 01> lsv_cut_bddsize 3
5: 5: 2
5: 1 2: 3
6: 6: 2
6: 2 3: 3
7: 7: 2
7: 5 6: 3
7: 2 3 5: 4

```

7: 1 2 6: 4
7: 1 2 3: 3

To achieve this function, you can use the BDD *Cudd_DagSize* operator to get the BDD size.

Notice. For problems 4.1 and 4.2, you may see some built-in functions to perform building BDD or truth table for cuts. You are allowed to refer to them, but you have to write your own procedures. Directly calling or copying from the built-in functions will be viewed as plagiarism.

Files to Submit

1. The BLIF file in problem 2(a).
2. The PDF report named **report.pdf**.
3. The source codes in problem 4 (by pull request).